SC3000 Artificial Intelligence
Assignment 2

Learning to Use Prolog as a Logic Programming Tool

Team: [team name] · Tutorial group: [e.g. TSR1]

Name	Matriculation No.	Contribution
[name]	[matric no.]	[contribution]
[name]	[matric no.]	[contribution]
[name]	[matric no.]	[contribution]

All traces were produced with SWI-Prolog 10.0.2 by typing `trace.` before the query and pressing <enter> (creep) at every port.

Exercise 1: Research Data Sharing

1.1 First-Order Logic

Statement	FOL
A research group called visionlab owns a dataset called medset.	$\text{ResearchGroup}(\text{visionlab}) \land \text{Dataset}(\text{medset}) \land \text{Owns}(\text{visionlab}, \text{medset})$
The dataset contains patient data.	$\text{ContainsPatientData}(\text{medset})$
anna is an intern at visionlab.	$\text{Intern}(\text{anna}, \text{visionlab})$
anna uploaded medset to a public repository.	$\text{UploadedToPublicRepo}(\text{anna}, \text{medset})$
Any dataset containing patient data is considered sensitive.	$\forall d\,[\text{Dataset}(d) \land \text{ContainsPatientData}(d) \rightarrow \text{Sensitive}(d)]$
An intern is not authorized to publish sensitive datasets owned by their research group.	$\forall x\,\forall g\,\forall d\,[\text{Intern}(x, g) \land \text{Owns}(g, d) \land \text{Sensitive}(d) \rightarrow \neg\text{Authorized}(x, d)]$
Uploading a dataset to a public repository is considered publishing the dataset.	$\forall x\,\forall d\,[\text{UploadedToPublicRepo}(x, d) \rightarrow \text{Publishes}(x, d)]$
Anyone who publishes a sensitive dataset without authorization violates the research data policy.	$\forall x\,\forall d\,[\text{Publishes}(x, d) \land \text{Sensitive}(d) \land \neg\text{Authorized}(x, d) \rightarrow \text{ViolatesPolicy}(x)]$

Intern(x, g) reads “x is an intern at research group g”, and Authorized(x, d) reads “x is authorized to publish d”.

1.2 Prolog clauses (`team_qn_1_2.pl`)

A Horn clause cannot have a negated head, so ¬Authorized(x, d) becomes the positive predicate `not_authorized(X, D)`. We did not use negation as failure (`\+ authorized(X, D)`), because then the conclusion would rest on a missing fact rather than on the stated intern rule.

```
% Exercise 1.2: Research Data Sharing
% Query: violates_policy(anna).

research_group(visionlab).
dataset(medset).
owns(visionlab, medset).
contains_patient_data(medset).
intern(anna, visionlab).
uploaded_to_public_repo(anna, medset).

sensitive(D) :- dataset(D), contains_patient_data(D).
not_authorized(X, D) :- intern(X, G), owns(G, D), sensitive(D).
publishes(X, D) :- uploaded_to_public_repo(X, D).
violates_policy(X) :- publishes(X, D), sensitive(D), not_authorized(X, D).
```

1.3 Proof that anna violates the research data policy

Prolog proves the query by refutation. It adds the negated goal ¬ViolatesPolicy(anna) and resolves it against the clauses until it reaches the empty clause. The goal unifies with the head of the `violates_policy` rule, which gives three subgoals:

1. `publishes(anna, D)` resolves with the publishing rule and then the upload fact, binding **D = medset**.
2. `sensitive(medset)` resolves with the sensitivity rule, then the facts `dataset(medset)` and `contains_patient_data(medset)`.
3. `not_authorized(anna, medset)` resolves with the intern rule: `intern(anna, G)` binds G = visionlab, then `owns(visionlab, medset)` and `sensitive(medset)` are proved again.

Every subgoal is resolved, so the empty clause is derived and `violates_policy(anna)` is **true**.

```
?- trace.
true.

[trace] ?- violates_policy(anna).
   Call: (12) violates_policy(anna) ? creep
   Call: (13) publishes(anna, _14444) ? creep
   Call: (14) uploaded_to_public_repo(anna, _14444) ? creep
   Exit: (14) uploaded_to_public_repo(anna, medset) ? creep
   Exit: (13) publishes(anna, medset) ? creep
   Call: (13) sensitive(medset) ? creep
   Call: (14) dataset(medset) ? creep
   Exit: (14) dataset(medset) ? creep
   Call: (14) contains_patient_data(medset) ? creep
   Exit: (14) contains_patient_data(medset) ? creep
   Exit: (13) sensitive(medset) ? creep
   Call: (13) not_authorized(anna, medset) ? creep
   Call: (14) intern(anna, _24168) ? creep
   Exit: (14) intern(anna, visionlab) ? creep
   Call: (14) owns(visionlab, medset) ? creep
   Exit: (14) owns(visionlab, medset) ? creep
   Call: (14) sensitive(medset) ? creep
   Call: (15) dataset(medset) ? creep
   Exit: (15) dataset(medset) ? creep
   Call: (15) contains_patient_data(medset) ? creep
   Exit: (15) contains_patient_data(medset) ? creep
   Exit: (14) sensitive(medset) ? creep
   Exit: (13) not_authorized(anna, medset) ? creep
   Exit: (12) violates_policy(anna) ? creep
true.
```

Exercise 2: Software Dependency Reasoning

2.1 Knowledge base and recursive rule (`team_qn_2_1.pl`)

```
% Exercise 2.1: Software Dependency Reasoning
% Queries: requires(visionapp, P).        % press ; to list every package
%          requires(visionapp, driver).

depends_on(visionapp, pytorch).
depends_on(visionapp, opencv).
depends_on(pytorch, cuda).
depends_on(opencv, numpy).
depends_on(cuda, driver).

requires(X, Y) :- depends_on(X, Y).
requires(X, Y) :- depends_on(X, Z), requires(Z, Y).
```

The first clause is the base case (a direct dependency). The second clause is the recursive case: X depends on some Z, and Z requires Y. Pressing `;` returns every package that visionapp requires:

```
?- requires(visionapp, P).
P = pytorch ;
P = opencv ;
P = cuda ;
P = driver ;
P = numpy ;
false.
```

So **visionapp requires pytorch, opencv, cuda, driver and numpy**.

Trace for `requires(visionapp, driver)`:

```
?- trace.
true.

[trace] ?- requires(visionapp, driver).
   Call: (12) requires(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, driver) ? creep
   Fail: (13) depends_on(visionapp, driver) ? creep
   Redo: (12) requires(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _17230) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, driver) ? creep
   Fail: (14) depends_on(pytorch, driver) ? creep
   Redo: (13) requires(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _22554) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires(cuda, driver) ? creep
   Call: (15) depends_on(cuda, driver) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Exit: (14) requires(cuda, driver) ? creep
   Exit: (13) requires(pytorch, driver) ? creep
   Exit: (12) requires(visionapp, driver) ? creep
true .
```

driver is not a direct dependency of visionapp, so the first clause fails and Prolog backtracks into the recursive clause. That clause follows visionapp → pytorch, where the direct check fails again, and then pytorch → cuda, where `depends_on(cuda, driver)` succeeds. This proves the chain visionapp → pytorch → cuda → driver. The answer `true .` means a choice point was still open (for example the opencv branch), and we pressed <enter> instead of asking for more proofs.

2.2 Effect of clause order (`team_qn_2_2.pl`)

Both versions sit in one file under different names, `requires_a` (Version A) and `requires_b` (Version B), so they can be compared on the same facts.

```
% Exercise 2.2: Clause order and cyclic dependencies
% Queries: requires_a(visionapp, driver).
%          requires_b(visionapp, driver).

depends_on(visionapp, pytorch).
depends_on(visionapp, opencv).
depends_on(pytorch, cuda).
depends_on(opencv, numpy).
depends_on(cuda, driver).
% depends_on(driver, pytorch).   % uncomment for the cyclic dependency part

% Version A
requires_a(X, Y) :- depends_on(X, Y).
requires_a(X, Y) :- depends_on(X, Z), requires_a(Z, Y).

% Version B
requires_b(X, Y) :- depends_on(X, Z), requires_b(Z, Y).
requires_b(X, Y) :- depends_on(X, Y).
```

Version A (base clause first):

```
?- trace.
true.

[trace] ?- requires_a(visionapp, driver).
   Call: (12) requires_a(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, driver) ? creep
   Fail: (13) depends_on(visionapp, driver) ? creep
   Redo: (12) requires_a(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _17266) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires_a(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, driver) ? creep
   Fail: (14) depends_on(pytorch, driver) ? creep
   Redo: (13) requires_a(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _22590) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires_a(cuda, driver) ? creep
   Call: (15) depends_on(cuda, driver) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Exit: (14) requires_a(cuda, driver) ? creep
   Exit: (13) requires_a(pytorch, driver) ? creep
   Exit: (12) requires_a(visionapp, driver) ? creep
true .
```

Version B (recursive clause first):

```
?- trace.
true.

[trace] ?- requires_b(visionapp, driver).
   Call: (12) requires_b(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _14602) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires_b(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _17262) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires_b(cuda, driver) ? creep
   Call: (15) depends_on(cuda, _19922) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Call: (15) requires_b(driver, driver) ? creep
   Call: (16) depends_on(driver, _22582) ? creep
   Fail: (16) depends_on(driver, _22582) ? creep
   Redo: (15) requires_b(driver, driver) ? creep
   Call: (16) depends_on(driver, driver) ? creep
   Fail: (16) depends_on(driver, driver) ? creep
   Fail: (15) requires_b(driver, driver) ? creep
   Redo: (14) requires_b(cuda, driver) ? creep
   Call: (15) depends_on(cuda, driver) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Exit: (14) requires_b(cuda, driver) ? creep
   Exit: (13) requires_b(pytorch, driver) ? creep
   Exit: (12) requires_b(visionapp, driver) ? creep
true .
```

Prolog tries the clauses of a predicate from top to bottom and the subgoals of a body from left to right. It searches depth-first and backtracks on failure. The clause order therefore decides which branch of the search tree is explored first:

- **Version A** first asks “is Y a *direct* dependency of X?” at each package, and only goes one level deeper if that fails. The trace checks visionapp (no), then pytorch (no), then cuda (yes). It finds the proof in 18 steps and never visits driver’s own dependencies.
- **Version B** descends first. It follows the first dependency at each level all the way to the bottom (visionapp → pytorch → cuda → driver) before trying any base clause. At driver both clauses of `requires_b(driver, driver)` fail, because driver has no dependencies. Prolog then backtracks to `requires_b(cuda, driver)` and tries its second (base) clause, which succeeds. That detour (the extra Call/Fail/Redo at depth 15–16) makes the proof 22 steps.

Both versions have the same declarative meaning, so they prove the same facts. Only the procedural proof sequence changes. The order of the answers changes too: Version A returns shallow dependencies first, and Version B returns the deepest first.

```
?- requires_a(visionapp, P).
P = pytorch ;
P = opencv ;
P = cuda ;
P = driver ;
P = numpy ;
false.

?- requires_b(visionapp, P).
P = driver ;
P = cuda ;
P = numpy ;
P = pytorch ;
P = opencv.
```

Adding `depends_on(driver, pytorch).`

This fact closes a cycle: pytorch → cuda → driver → pytorch. The problem is **non-termination (infinite recursion)**. The recursive clause calls itself on pytorch, cuda, driver, pytorch, … forever. Prolog's depth-first search does not remember which goals it is already trying to prove (there is no loop check), so the search tree now contains an infinite branch. Results with the cycle fact uncommented:

Version A. The first answer to `requires_a(visionapp, driver)` is still found, because the base clause hits driver before the cycle is entered. Backtracking, however, goes round the cycle forever. Asking for all packages gives an endless stream of duplicates, and numpy is never reached because the pytorch branch never finishes. A query that should be false, such as `requires_a(cuda, opencv)`, never terminates. Here the recursive call is the last, deterministic call, so SWI-Prolog's last-call optimisation runs the loop in constant memory. It spins forever without any error, so we stopped it with a time limit:

```
?- requires_a(visionapp, P).
P = pytorch ;
P = opencv ;
P = cuda ;
P = driver ;
P = pytorch ;
P = cuda ;
P = driver ;
P = pytorch ;
P = cuda ;
P = driver ;
P = pytorch ;
P = cuda ;
P = driver .

?- call_with_time_limit(10, requires_a(cuda, opencv)).
ERROR: Unhandled exception: Time limit exceeded
```

Version B. Even the true query `requires_b(visionapp, driver)` gets no answer. Because the recursive clause is always tried first, Prolog keeps following cuda → driver → pytorch → cuda → … and never backtracks to a base clause. The trace repeats the same three calls at ever greater depth. Each level leaves a choice point for the untried base clause, so the stack grows until SWI-Prolog

raises a stack-overflow error. With the default 1 GB limit it was still running after 5 minutes, so we ran it with `--stack-limit=64m`, where it fails in seconds.

```
?- trace.
true.

[trace] ?- requires_b(visionapp, driver).
   Call: (12) requires_b(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _14602) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires_b(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _17262) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires_b(cuda, driver) ? creep
   Call: (15) depends_on(cuda, _19922) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Call: (15) requires_b(driver, driver) ? creep
   Call: (16) depends_on(driver, _22582) ? creep
   Exit: (16) depends_on(driver, pytorch) ? creep
   Call: (16) requires_b(pytorch, driver) ? creep
   Call: (17) depends_on(pytorch, _25242) ? creep
   Exit: (17) depends_on(pytorch, cuda) ? creep
   Call: (17) requires_b(cuda, driver) ? creep
   Call: (18) depends_on(cuda, _27902) ? creep
   Exit: (18) depends_on(cuda, driver) ? creep
   Call: (18) requires_b(driver, driver) ? creep
   Call: (19) depends_on(driver, _30562) ? creep
   Exit: (19) depends_on(driver, pytorch) ? creep
   Call: (19) requires_b(pytorch, driver) ? creep
   Call: (20) depends_on(pytorch, _33222) ? creep
   Exit: (20) depends_on(pytorch, cuda) ? creep
   Call: (20) requires_b(cuda, driver) ? abort

?- requires_b(visionapp, driver).
ERROR: Stack limit (64.0Mb) exceeded
ERROR:   Stack sizes: local: 45.3Mb, global: 2.3Mb, trail: 0Kb
ERROR:   Stack depth: 296,753, last-call: 0%, Choice points: 296,746
ERROR:   Possible non-terminating recursion:
ERROR:     [296,752] user:requires_b(pytorch, driver)
ERROR:     [296,751] user:requires_b(driver, driver)
   Exception: (296,752) requires_b(pytorch, driver) ? abort
```

The cause is the cycle in the data combined with Prolog's depth-first, loop-unaware proof procedure. Reordering the clauses only changes whether an answer happens to be found before the infinite branch is entered. Neither order makes the search terminate. A terminating version would have to remember visited packages (for example, pass a visited list and check `\+ member(Z, Visited)`), or use SWI-Prolog tabling (`:- table requires/2.`).